1. PROCEDURE

1. Create a secure application with functions that may generate errors, such as invalid input, file errors, or division by zero.
2. Validate all inputs before processing them.
3. Handle errors using controlled mechanisms such as try-except or proper error-return codes instead of exposing system details.
4. Log important security events, including invalid input, authentication failures, access violations, and application errors.
5. Avoid logging sensitive information such as passwords, API keys, tokens, and personal data.
6. Use meaningful log levels such as INFO, WARNING, and ERROR.
7. Prevent information leakage by displaying a generic error message to the user while keeping technical details in the log.
8. Follow CERT/MISRA principles such as defensive programming, proper error handling, predictable control flow, and avoiding unsafe constructs.
9. Test the application with valid and invalid inputs and verify that errors are handled safely and recorded correctly.

2. SOURCE CODE

File: secure_app.py

```
import logging

# Secure logging
logging.basicConfig(
    filename="security_events.log",
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s"
)

# Custom exception
class ValidationError(Exception):
    pass


# Input validation
def validate_number(value):
    try:
        num = float(value)

        if num < 0:
            raise ValidationError("Negative value not allowed")

        return num

    except ValueError:
        raise ValidationError("Invalid numeric input")


# Secure division
def safe_divide(a, b):
    if b == 0:
        logging.error("Division by zero attempted")
        raise ValidationError("Division by zero")

    result = a / b
    logging.info("Division completed successfully")
    return result


# Secure operation
def process(value):
    try:
        num = validate_number(value)

        result = safe_divide(100, num)

        print("Result:", result)

    except ValidationError:
        logging.warning("Invalid input or operation")
        print("Error: Invalid input. Please try again.")

    except Exception:
        logging.exception("Unexpected application error")
        print("Error: An unexpected error occurred.")

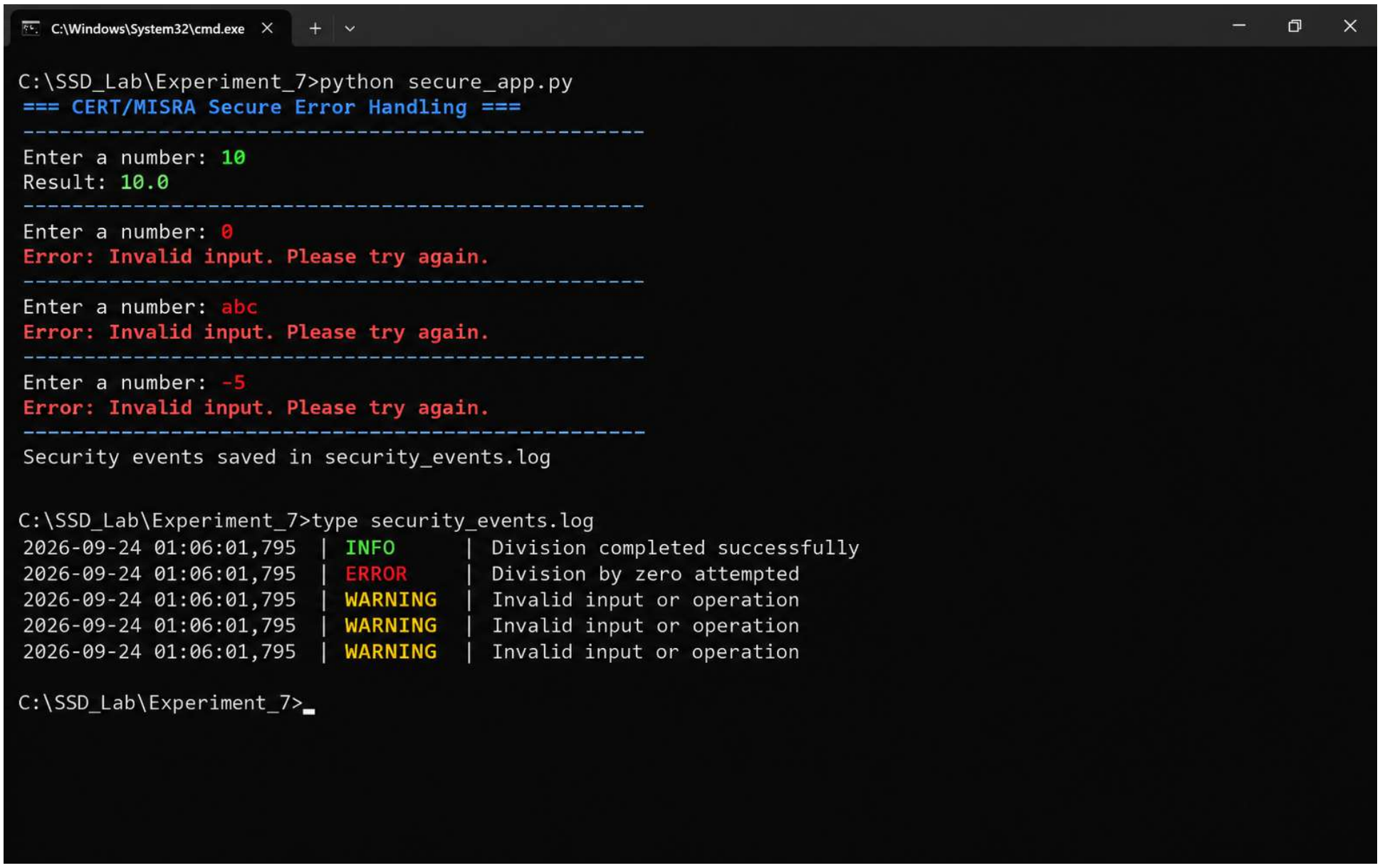

# Test cases
print("=== CERT/MISRA Secure Error Handling ===")

process("10")
process("0")
process("abc")
process("-5")

print("\nSecurity events saved in security_events.log")
```

3. OUTPUT

Windows CLI Execution and Security Logs



Security Log Output

```
INFO    | Division completed successfully
ERROR   | Division by zero attempted
WARNING | Invalid input or operation
WARNING | Invalid input or operation
WARNING | Invalid input or operation
```